

1 кандидат

Детализация результата:

Видеоинтервью

6.67

Пояснение:

Этот кандидат продемонстрировал общее понимание ключевых технологий и процессов, связанных с вакансией Middle+ Python Developer, включая работу с Kafka, PostgreSQL, CI/CD и асинхронным кодом. Ответы содержат технические знания и опыт, однако часто не хватает конкретных примеров, метрик и детального описания личного вклада, что затрудняет оценку глубины компетенций на уровне Senior. Кандидат проявляет мотивацию к развитию и понимание важности интеграций и автоматизации.

Сильные стороны:

- Понимание работы с Apache Kafka и паттерна Outbox для надежной обработки сообщений.
- Знание оптимизации запросов в PostgreSQL и использования explain для анализа производительности.
- Опыт настройки CI/CD пайплайнов в GitLab с управлением Docker-образами и откатами.
- Понимание принципов асинхронного программирования в Python и различий между FastAPI и Django.
- Использование ИИ для автоматизации рутинных задач и анализа проектов.

Области для роста:

- Недостаток конкретных примеров и метрик, подтверждающих личный вклад и результаты.
- Общее описание процессов без детального технического раскрытия и инструментов.
- Не всегда четкое структурирование ответов и акцент на результатах.
- Мало информации о масштабах проектов и объемах данных.
- Ограниченное описание практического применения ИИ и его влияния на эффективность работы.

Навыки, выявленные во время интервью

Профессиональные навыки:

- API
- Apache Kafka
- PostgreSQL
- ETL
- CI/CD
- Docker
- Python
- FastAPI
- Django
- ИИ

Гибкие навыки:

- Мотивация к развитию
- Внимание к деталям
- Аналитическое мышление
- Ответственность
- Техническое мышление
- Коммуникация

Антифрод: действия при прохождении интервью

Переход на другое окно: 1

Копирование текста: 0

Расшифровка ответов:

Оценка видеоинтервью

6.67

Расскажите почему вы сейчас находитесь в поиске работы и что для себя ищите? Каким проектом и какими задачами хотелось бы заниматься?

Так, сейчас нахожусь в поиске работы, потому что в данный момент на позиции, где я сейчас нахожусь, достиг некой точки. Точки именно по финансам, по росту и так далее. То есть хочется развиваться, но пока стоп. Каким проектам хочется развиваться, заниматься? Главное, чтобы были интеграции. Интеграции – это всегда интересно. Интеграции – это построение каких-то контрактов, выполнение контрактов, работа с внешними системами, это API-шины, ещё многое разное. Многие протоколы. То есть интеграция – это интересно.

Расскажите про свой продакшн-сервис на Python с очередью — Kafka или RabbitMQ. Какие библиотеки, как добивались, чтобы сообщение не потерялось и не обработалось дважды, и что делали с необработанными сообщениями?

Использовали Kafka на продакшене, там сообщения идут, как минимум сообщения хранятся в самой Kafka, дублирование там вроде настроено было девопсами за счет того, что у каждого сообщения есть в принципе идентификатор, GUID какой-то, плюс если мы говорим про отправку, то есть паттерн Outbox, то мы гарантируем, что сообщение отправится, вот, и в обратную сторону, так, ну, был Consumer, ну, получается, сама Kafka была организована так, что там, в принципе, хранилка была, и сообщения уже с были бы были с необработанными, так, что делать с необработанными сообщениями, так, ну, там, по сути, был всегда offset, то есть там был Consumer настроенный, и, по сути, это offset учитывает свежие сообщения, то есть если даже остановился Consumer, он все равно вычитет с того места, где закоммитился offset, не было такого, наверное, ну, было такое, конечно, непрочитанное, но необработанные сообщения, но это решалось сразу же после включения Consumer.

Как вы загружали большие объёмы данных в PostgreSQL? Назовите объёмы, как была устроена загрузка и как находили причину медленных запросов?

Так, загружали большие объемы данных в постгре. Если мы говорим про insert и прям огромные размеры, то конечно для инсертгов есть такие штуки как ClickHouse, вертикальные базы данных. А вообще если приходится работать с постгре, то надо в какую-то очередь ставить, потому что insert — это дорогая операция длительная. И таблица лочится, то есть по идее там систему надо смотреть. Опять же, нельзя подобрать именно какой-то один корректный ответ. Так, загрузка была. В целом всё, таких огромных объёмов, что прям что-то тормозило и сильно лочилось, не было. Но опять же, были обработчики, был тот же паттерн Outbox, то есть, например, если надо было, чтобы быстро что-то записалось, и потом происходило какое-то действие, мы бы формировали какой-то GUID, и по нему дальше отслеживали какие-то действия в дальнейшем. Так, причины медленных запросов – это чаще всего, чаще всего мы просто смотрели на модели и как сделаны модели, где проставлены индексы, почему у нас там идёт работа. Возможно, формировали бы составные индексы, что вроде constraint. Но если совсем всё плохо, ну то есть, соответственно, если мы находили причину, делали индексы, делали префетчи, делали ещё что-нибудь, там select related и всё такое, ну чаще всего префетчами докидывали. Но если прям медленный запрос, то уже работали на уровне контейнера и через explain анализ смотрели, там, как проходит запрос, сколько loops, какой фактический ответ, какую база подбирала, точнее, с каких строк она убирала, хотела выбрать, и планово, и реальный результат. Вот, уже через explain. Но это очень редко было, что прям такие запросы.

Расскажите про CI/CD-пайплайн для Python-сервиса в контейнерах, который настраивали сами. Что делал пайплайн, как собирали Docker-образ, как деплоили и откатывались?

Так, про CICD Pipeline. Это, в принципе, были джобы, написанные для GitLab. Ну, вообще весь CICD процесс был в GitLab. Вот, то есть там просто джобы, был какой-то общий пайп, в котором запускались какие-то джобы, типа билд, тесты прогоняться должны были, и деплой, например, на стенд, на стенд 2, там стенд для тестирования, стенд фича-стенд какой-нибудь был. Ну, у нас был. Вот. Собирали докер-образ. Там, да, в принципе, джоба по пайпу, допустим, для билда, она была, удаляла все имейджи старые, потому что уже, ну, было такое, что столкнулся с проблемой, что засорялась память, вот, имейджей много насоздавалось, и там приходилось очищать. Но, в принципе, это несложно оказалось. Билд образа. И, в принципе, всё. По откату. Также джобы. Там вот как раз можно было сохранить старый имейдж, у нас такое было уже, и на ту версию откатывали. То есть просто брали тот имейдж, собранный докером, который сформировался в прошлый раз. Вот. Ну, это просто были проекты, где были

откаты, где не было откатов, там всё другими средствами решалось, не откатывало он, не откатами пайпов, допустим, на продажах или ещё где-то.

Когда выбираете асинхронный код, а когда синхронный? На примере из своего проекта. И что будет, если внутри асинхронного кода вызвать блокирующую функцию?

Так, асинхронные – это когда у нас есть какие-то коннекты к базе данных, то есть это все быстрее делать через асинхронщину. То есть там есть event loop, который переходит от await к await, грубо говоря. Вот. И мы ожидаем какой-то результат, то есть во время, когда база, например, что-то вызывает, мы это можем не ждать. В момент, когда мы это получим, event loop перебросит нас на другую часть кода, которая будет выполняться асинхронно. Вот в этом, когда я буду синхронный, когда есть четкая последовательность действий, наверное, просто когда какая-то процедура, в целом это просто одна функция, она выполняется синхронно. Когда есть некий протокол, когда некий флоу, некая последовательность, хотя, я думаю, это не об этом вопрос. Это все можно сделать асинхронно. Так. На примере проекта мы на Django работали. Django-ниде, там не было асинхронного. То есть я просто с ним работал с асинхронщиной, но не в рамках проектов, это в рамках проектов тоже. Это было на FastAPI, и это подключение к BD, это там простые... там вьюхи были, и там, в принципе, все подключения к BD и зависимости, там было все асинхронное. На Django нет. В большинстве случаев был код синхронный. Что будет внутри синхронного кода? Блокирующая функция. Блокирующую я просто не могу понять немного корректность вопроса. Если мы в асинхронном коде вызываем синхронный, то мы теряем свою синхронность. То есть он выполнится весь код синхронно. То есть не будет никаких ожиданий, там await-ов и так далее.

Расскажите как используете ИИ в своей работе?

Так, ну и чаще всего это бывает, если вот, например, как у нас было у клиента. Ну, во-первых, ИИ для рутинной работы – это написание тестов. То есть если задал ему стиль, задал какие-то параметры, как это делать, какой-то style guide, то тесты пишутся очень быстро. И второе, очень важное – это когда у клиента отгружают какие-то микросервисы, которые делают какой-то функционал, и клиент ничего не говорит, например, долго ждём ответа. Быстрее самому клонировать этот репозиторий, посмотреть на проект, залить его в ИИ. Ну, там не совсем... в общем, использовать ИИ, чтобы да, он разобрал проект быстрее. Просто там, опять же, это всё немного согласуется с самим клиентом. То есть, ну, мы не могли просто сами взять и запихнуть всё в ИИ. Там всё через согласование. То есть можно ли.

Нам не отвечают, например, разработчики клиента, или долго эта коммуникация происходит. Мы делаем быстрый запрос, нам отвечают, ладно, окей. Ну, уже было такое, что мы использовали этот момент. ИИ быстро рассказал, что происходит в проекте, шаг за шагом, какой контракт формируется, например. Вот. То есть, такой момент. Там не проблема была ни в Swagger, ни в API, а именно в моделях, как хранились данные в базе, и как формировались данные для сохранения в эту модельку. В общем, ИИ помогал.

ВАРИАНТ ОБРАТНОЙ СВЯЗИ ДЛЯ КАНДИДАТА:

По итогам интервью видно, что у вас хорошая техническая база и понимание основных инструментов и процессов, с которыми вы работаете. Вы уверенно ориентируетесь в Kafka, PostgreSQL, CI/CD и асинхронном программировании на Python, а также достаточно чётко понимаете, в каком направлении хотите развиваться дальше.

При этом в ответах иногда не хватало конкретики из реального опыта. Было бы полезно подробнее рассказывать о проектах: какие задачи стояли перед вами, с какими объёмами данных и инструментами вы работали, за что отвечали лично и какого результата удалось добиться. Такие примеры позволяют лучше оценить не только технические знания, но и ваш реальный вклад в проект.

Также рекомендуем немного больше внимания уделять структуре ответов и техническим деталям, так ваш опыт и уровень экспертизы будут звучать более убедительно.

2 кандидат

Детализация результата:

Видеоинтервью

Пояснение:

Данный кандидат демонстрирует хороший уровень профессиональных знаний и практического опыта, соответствующий уровню Senior Python разработчика. Ответы содержат подробные технические детали и примеры, особенно в вопросах, связанных с обработкой сообщений, загрузкой данных и асинхронным программированием. Однако в некоторых ответах не хватает конкретных метрик и более глубокого раскрытия личного вклада, что является зоной для развития.

Сильные стороны:

- Глубокое понимание работы с очередями сообщений и обеспечением надежности (RabbitMQ, подтверждения, retry, dead-letter).
- Хорошее знание PostgreSQL, включая загрузку больших объемов данных и оптимизацию запросов с использованием explain analyze.
- Практический опыт реализации асинхронного кода на Python с пониманием последствий блокирующих вызовов.
- Ответственное и осознанное использование ИИ для поддержки разработки и проверки кода.
- Понимание и опыт работы с CI/CD пайплайнами, Docker и Kubernetes.

Области для роста:

- Недостаток конкретных метрик и результатов, подтверждающих эффективность выполненных действий.
- Мало примеров личного вклада и технических деталей в настройке CI/CD пайплайна.
- Отсутствие упоминания опыта работы с Kafka, хотя вопрос включал и Kafka, и RabbitMQ.
- Недостаточно конкретики в описании использования ИИ и инструментов для этого.
- В ответах иногда присутствуют общие формулировки без глубокого раскрытия задач и достижений.

Навыки, выявленные во время интервью

Профессиональные навыки:

- Python
- RabbitMQ
- PostgreSQL
- CI/CD
- Docker
- Kubernetes
- ИИ в разработке

Гибкие навыки:

- Саморазвитие
- Мотивация
- Внимание к деталям
- Ответственность
- Аналитическое мышление
- Командная работа
- Проблемное мышление
- Критическое мышление

Антифрод: действия при прохождении интервью

Переход на другое окно: 10

Копирование текста: 0

Оценка видеоинтервью

8

Расскажите почему вы сейчас находитесь в поиске работы и что для себя ищите? Каким проектом и какими задачами хотелось бы заниматься?

В данный момент нахожусь в поиске работы, потому что мой последний проект, которым я занимался – именно реализация финтех-платформы для алгоритмической торговли – этот проект был завершён. То есть идея, вокруг которой он строился, была исчерпана, поэтому пришло время двигаться дальше. Хотелось бы заниматься задачами, проектами, которые соответствуют моему опыту, предполагалось бы какое-то развитие, изучение нового, разнообразные задачи и прочее.

Расскажите про свой продакшн-сервис на Python с очередью — Kafka или RabbitMQ. Какие библиотеки, как добивались, чтобы сообщение не потерялось и не обработалось дважды, и что делали с необработанными сообщениями?

Если говорить про RabbitMQ, то использовал Fastream именно для обработки сообщений. И чтобы сообщение не потерялось, мы при отправке сообщений продюсерам ждём подтверждения. А чтобы при получении не обработалось дважды, нам нужно, во-первых, использовать подтверждение обработки сообщения. А если вдруг у нас какая-то обработка сложная и предполагает

множество этапов, то тогда идемпотентность обработки достигается за счёт того, что мы фиксируем где-то успешные этапы прохождения. И в случае повторного получения сообщения мы сверяемся с этой табличкой, то есть это может быть табличка какая-то, например, в Postgres. И пропускаем успешно пройденные этапы, и у нас нет повторной обработки, и подтверждаем обработку сообщения. Таким образом, у нас сообщение обрабатывается единожды. Если же вдруг сообщение у нас обрабатывается с ошибкой, то тут можно построить такую схему ретрая очередей. Это вот применительно к RabbitMQ, где у нас сообщения будут вываливаться в очередь, повторно возвращаться в изначальную очередь с какой-нибудь экспоненциальной задержкой. Например, три очереди – там, первая ретри, вторая ретри, третья ретри с экспоненциальной задержкой. И в случае трёх неуспешных попыток может быть dead-letter очередь, где уже мы будем видеть, что сообщение несколько раз было неуспешно обработано, посмотрим причину ошибки и можем повторно его запустить в изначальную очередь. То есть при этом забыл, что хотел сказать... А, именно при вот этом механизме ретри и dead-letter мы можем использовать заголовки для RabbitMQ для того, чтобы считать количество попыток, чтобы корректно маршрутизировать именно ретри очереди с сообщением.

Как вы загружали большие объёмы данных в PostgreSQL? Назовите объёмы, как была устроена загрузка и как находили причину медленных запросов?

Но большие объёмы данных загружались через файлы, то есть можно загружать файл на сервер и его заливать уже в таблицу. Объёмы, объёмы, объёмы — там были файлы на несколько гигабайт. Если речь про код, то, соответственно, тут, когда у нас предполагается какой-то большой объём для загрузки, то тут не единичные запросы, вставки, отсёрты, апдейты или апсёрты, то тут предполагается батчевая вставка. Когда мы смотрим на наш объём данных, разбиваем его на какие-то батчи, условно по тысячу строк, и заливаем его вот такими запросами. А если говорить про медленные запросы, тут всегда мы смотрим на explain. Причём, когда мы выполняем explain, нужно сначала посмотреть, именно выполнить explain analyze, чтобы посмотреть, что планировщик ожидает количество строк с планировщиком и реальное количество строк в запросе совпадает. То есть это говорит о том, что у планировщика актуальная статистика на руках. А если же вдруг она не бьётся, то нужно сначала всё-таки переанализировать табличку, чтобы была актуальная статистика, после этого опять посмотреть explain и уже смотреть на то, какие узлы мы видим в схеме запросов, их анализировать. То есть причин может быть разные: неактуальные джойны, неактуальные группировки, не хватает воркмема для обработки запросов, там и выполнение операции в памяти, вываливается что-то на диск. Либо же нужно переписать как-то немножко запрос, либо чтобы уменьшить выборку, либо уже

там смотреть сторону каких-то материализованных представлений. То есть варианты оптимизации запросов бывают разные, абсолютно.

Расскажите про CI/CD-пайплайн для Python-сервиса в контейнерах, который настраивали сами. Что делал пайплайн, как собирали Docker-образ, как деплоили и откатывались?

Ну, CI/CD pipeline для Python-сервисов мы настраивали в GitLab. Есть у нас какой-то сервис готовый, и мы настраиваем для него CI/CD, который заключается в следующем: если мы отправляем реквест в репозиторий, то у нас запускаются этапы CI/CD, где мы проверяем линтерами код, запускаем тесты. Если все это — основные этапы CI, если все ок, то запускаются этапы CD, где мы пересобираем контейнер, и он уже отправляется в регистры. И там дальше были некоторые этапы, которые настраивал DevOps, я про них ничего не могу сказать, потому что мы ... там дальше был Kubernetes, и мы вот только некоторые правки в конфиге Kubernetes вносили, и вот контролировали в дашбордах, как у нас там поды перезапускаются, что они успешно перезапустились, где-то некоторые опции правили и так далее. А как откатывались? Ну, соответственно, нужно либо применить откатывающий реквест (полностью новый), либо же сделать откат реквеста в самом GitLab.

Когда выбираете асинхронный код, а когда синхронный? На примере из своего проекта. И что будет, если внутри асинхронного кода вызвать блокирующую функцию?

Ну, асинхронный код выбирает в основном, когда у нас много IO операций, есть соответствующие библиотеки для того, чтобы работать в асинхронном варианте. Асинхронный код больше подходит, когда у нас нет IO задачи, какие-то CPU задачи. Но это если мы не говорим про какое-то разделение на потоки и процессы. Вот, например, из своего проекта. А, понял. Например, из своего проекта могу рассказать. Реализовывал задачу обработки торговых операций, и у меня там было очень много IO операций, которые были связаны с тем, что я получал внешние данные какие-то от брокера, взаимодействовал с базой. То есть тут очень много IO операций, очень мало вычислений, очень много обращений к каким-то источникам. И, соответственно, тут подойдет очень асинхронный код, где у меня был один поток в рамках одного процесса, где был событийный цикл, и он обрабатывал вот соответствующие корутины. Корутины выполнялись конкурентно, все пакеты, которые использовались, были в асинхронных вариантах, то есть не блокировали событийный цикл. И, соответственно, все это хорошо работало, ничего не блокировало работу потока,

кода. Вот. А вот что будет, если внутри асинхронного кода вызвать блокирующую функцию? Ну, соответственно, произойдет блокировка событийного цикла. То есть у нас есть какой-то код, который выполняет какую-то задачу, при этом при всем в нем нет await. Соответственно, управление событийным циклом не передается, корутина продолжает выполняться, блокируется событийный цикл, и никакие другие корутины не выполняются. Хотя внутри вот этого кода может быть какое-то синхронное обращение к БД, где в принципе вполне могли использовать какой-то синхронный вариант и дать возможность поработать другим корутинам.

Расскажите как используете ИИ в своей работе?

Но в своей работе ИИ я использую, конечно, он помогает ускорить разработку. В целом это выглядит следующим образом: если у меня есть какая-то задача, которую мне нужно реализовать, я даже, скорее всего, представляю, как её нужно правильно реализовать, но я сверяюсь обязательно с ИИ. То есть задаю какие-то точечные вопросы, слушаю варианты, где-то задаю какие-то уточняющие вопросы. В итоге у меня формируется какой-то микс своего мнения и рекомендаций ИИ – это вот относительно задач и их реализации. А потом, допустим, дополнительно где-то я сверяю какие-то куски кода, которые написаны мной, или же, например, прошу там ИИ сгенерить какой-то кусочек кода, обязательно разбираюсь в нём целиком, как он работает, чтобы именно понимать, что я копирую-вставляю. И, соответственно, также иногда свой какой-то код могу кусочек вставить в ИИ, посмотреть, что он мне подскажет, какие моменты я, может быть, упустил или что-то не так написал. Соответственно, вот так вот примерно выглядит. То есть проверяю и идеи для реализации задачи, и какие-то конкретные куски кода тоже проверяю. Ну и в целом на выходе получаю полное понимание того, как работает код и как я решаю задачу, не оставляю без внимания вот как это всё работает. Иначе можно бездумно что-то вставить, и потом какие-то моменты могут всплыть при работе кода.

ВАРИАНТ ОБРАТНОЙ СВЯЗИ ДЛЯ КАНДИДАТА:

По итогам интервью видно, что у вас хорошая техническая база и практический опыт работы с очередями сообщений, большими объёмами данных и асинхронным программированием. В ответах вы показали понимание того, как подходить к техническим задачам, и внимание к качеству кода и процессов.

При этом хотелось бы видеть больше конкретики о результатах вашей работы: какие задачи удалось решить, что изменилось после вашего участия и, по возможности, какие метрики это подтверждают. Также стоит подробнее

раскрывать именно свой вклад в проекты — за какие решения и участки работы вы отвечали лично.

Отдельно не хватило технических деталей по настройке CI/CD-пайплайнов и примеров использования современных ИИ-инструментов в работе. Более конкретные примеры в этих областях помогут полнее показать ваш практический опыт и уровень технической экспертизы.